# Q3 – Dictionaries: Farmer Helpline Complaint Analysis

## Frequency design

Tokenise complaint text consistently (lowercase and remove punctuation), ignore stop words stored in a set, and increment counts with dict.get or setdefault. Group complaints by district with a dictionary of lists. Sort by descending count and then alphabetically to resolve ties.

## Complexity

Dictionary updates and lookups are O(1) average, giving O(n) expected time for n tokens and O(d) space for d distinct words. Calling list.count for every distinct word scans the list d times and costs O(n·d). Counter implements frequency counting efficiently and clearly.

## Ordering and views

Python 3.7+ guarantees insertion order for dictionaries. keys(), values() and items() return dynamic view objects that reflect later dictionary changes. For ties, the key (-count, word) sorts highest counts first and alphabetically.

## Python Program

```
import re, time, random
calls = ["seeds not received seeds delayed", "fertiliser price high",
         "seeds price high subsidy not received"]
STOP = {"not", "high"}

def frequencies(texts, stop):
    counts = {}
    for text in texts:
        for word in re.findall(r"[a-z]+", text.lower()):
            if word not in stop:
                counts[word] = counts.get(word, 0) + 1
    return counts

def top(counts, k):
    return sorted(counts.items(), key=lambda item: (-item[1], item[0]))[:k]

if __name__ == "__main__":
    print(top(frequencies(calls, STOP), 3))
```